

GSoC 2026: Multimodal AI API Eval Framework (Dependency-Lite Architecture)

Organization: API Dash

Project Size: Large (350 hours)

1. Contact Information & Bio

Name: Mansi Tyagi

University: Lovely Professional University (B.Tech CSE, 6th Semester)

Email: mansityagi472@gmail.com

GitHub: <https://github.com/mansityagi01>

Bio:

I am a full-stack engineer and data architect specializing in Python machine learning pipelines and React-based frontends. I recently developed a Real Estate Market Forecasting tool using XGBoost and Python, and built full-stack applications like a React/Python Space News Aggregator. My foundation in algorithmic optimization (900+ competitive programming problems solved) ensures I can handle massive data streams efficiently. I am applying for this project because my experience in parsing heavy datasets and rendering them in responsive React dashboards aligns perfectly with the need for a lightweight, high-performance evaluation framework that doesn't freeze the user's UI during long benchmarking runs.

2. Project Abstract

API Dash needs a simple and unified way to benchmark multimodal AI models (text, image, voice) across different API providers. Tools like Lm-harness and lighteval already exist, but they depend a lot on CLI-based workflows, which makes them hard to manage and visualize properly. Also, running large benchmark suites locally can freeze normal REST-based web interfaces.

Following the maintainer's guidance, I will focus on proper Multimodal API Output testing instead of building complex agent orchestration.

I am proposing a simple "dependency-lite" setup with a Python backend using FastAPI and a frontend dashboard built with React and TypeScript.

The system will support both local file uploads and remote datasets (like HuggingFace or S3), stream benchmark logs in real-time using SSE (so there are no HTTP timeout issues), and use a zero-config SQLite database to store evaluation history. The goal is to keep everything easy to run with just a simple pip install and npm start, without any heavy setup.

3. Architectural Implementation

Part A: The Python Evaluation Engine (FastAPI):

- **Subprocess Wrappers:**
Instead of adding heavy tools like Celery or Redis, I will just use Python's built-in subprocess and asyncio to run Lm-harness and lighteval jobs. This keeps the setup very lightweight and easy to install.
- **SSE Streaming:**
Benchmarks can run for hours, so normal REST calls won't work (they'll time out). The FastAPI backend will use Server-Sent Events (SSE) to stream logs (stdout/stderr) and progress updates live to the React UI.
- **Multimodal Data Handling:**
I will create a single format to handle all data. It will support local file uploads (like CSV/JSON parsing in chunks) and also remote URLs, where images/audio are fetched only when needed so memory doesn't get overloaded.
- **State Management:**
I will use a simple SQLite database that auto-creates itself to track job status (Pending, Running, Completed, Failed) and store final results like Accuracy, BLEU, and ROUGE.

Part B: The React Dashboard (Frontend):

- **Evaluation Configurator UI:**
I will build a dynamic form where users can pick the modality (Text/Image/Voice), enter their API keys, choose the benchmark, and provide the dataset source.

- **Virtualized DOM for Heavy Traces:**
Large outputs (like 10,000+ rows of JSON) can crash normal React components. I will use virtualization (like react-window) so the Analysis tab stays smooth even with big results.
- **Live Execution Viewer:**
I will add a terminal-like UI that connects to the FastAPI SSE stream and shows live logs and progress updates while the benchmark is running.

4. Project Timeline (12 Weeks)

Community Bonding Period (May 1 - May 26):

- I will go through the current API Dash codebase and understand how everything is structured.
- Then I will finalize the exact JSON schemas for multimodal inputs (Text, Image, Audio) so there's a clear and consistent data contract between the React frontend and the FastAPI backend.
- After that, I will submit the initial "Idea Doc PR" to the API Dash repo to get the architecture reviewed and approved.

Phase 1: Backend Orchestration & Data Pipelines (Weeks 1 - 4):

- **Week 1:** I will set up the FastAPI backend and configure a simple SQLite database to track jobs.
- **Week 2:** I will build Python wrappers for Lm-harness and add SSE streaming endpoints so logs can be streamed live.
- **Week 3:** I will implement the multimodal data handling—parsing local CSV/JSON files and fetching images/audio from remote URLs when needed.
- **Week 4:** I will write proper backend unit tests and submit the Phase 1 Backend PR.

Phase 2: React UI & Real-Time Streaming (Weeks 5 - 8):

- **Week 5:** I will build the Evaluation Configurator forms—things like API key inputs, dataset selection, and parameter tuning in the UI.
- **Week 6:** I will connect the React frontend to the FastAPI SSE endpoints and create the live terminal/progress viewer.

- **Week 7:** I will add DOM virtualization so large JSON results can be rendered safely without slowing down the UI.
- **Week 8:** I will run full end-to-end testing using a standard text benchmark like MMLU and prepare for the mid-term evaluation.

Phase 3: Multimodal Expansion & Visualization (Weeks 9 - 12):

- **Week 9:** I will extend the framework to support Image (like captioning and VQA) and Voice (STT) benchmarks, not just text.
- **Week 10:** I will build a comparison dashboard-things like radar charts and latency vs accuracy tables to compare different AI APIs.
- **Week 11:** I will focus on performance-stress test large file uploads and make sure the UI stays responsive even with big datasets.
- **Week 12:** I will finish the documentation (setup guide and user manual), clean up the code, and submit the final PR for the complete framework.